

Pivot index

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return *the leftmost pivot index*. If no such index exists, return -1.

Example 1:

Input: `nums = [1,7,3,6,5,6]`

Output: 3

Explanation:

The pivot index is 3.

Left sum = `nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11`

Right sum = `nums[4] + nums[5] = 5 + 6 = 11`

Example 2:

Input: `nums = [1,2,3]`

Output: -1

Explanation:

There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: `nums = [2,1,-1]`

Output: 0

Explanation:

The pivot index is 0.

Left sum = 0 (no elements to the left of index 0)

Right sum = `nums[1] + nums[2] = 1 + -1 = 0`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int totalSum = 0;
        int leftSum = 0;

        for (int num : nums) {
            totalSum += num;
        }

        for (int i = 0; i < nums.size(); i++) {
            int rightSum = totalSum - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
};
```

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

```
nums =  
[1,7,3,6,5,6]
```

Output

3

Expected

3